

Architecture Decision Record

devex-platform — Golden Path Ecosystem

Date: June 2026 **Author:** Luis Cruz **Status:** Accepted

Context

Ten or more independent engineering teams need to operate under the same conventions — branch naming, commit messages, pipeline stages, infrastructure patterns — and report comparable DORA metrics, without a platform team becoming a bottleneck for every custom request. Today each team invents its own hooks, YAML, and CDK patterns, making cross-team measurement impossible and platform support unsustainable.

The solution is a two-component ecosystem: a **CLI** (*devex*) for local enforcement on engineer machines, and a **Framework** (*@luicruz01/devex-framework*) for CI/CD pipeline generation and reusable CDK constructs. Both share a single integration contract — the **Work ID** — and a fixed **DoraEvent** telemetry schema.

1. Architecture

The **CLI** runs on engineer machines, enforcing conventions locally: Work ID validation, git hook installation, and standards checks. Config resolves in priority order: CLI argument → DEVEX_WORK_ID env var → .devex/config.yaml.

The **Framework** runs in CI/CD, generating typed GitHub Actions workflows (PrPipelineGenerator, IntegrationPipelineGenerator) and CDK constructs (LambdaServiceConstruct) with built-in tagging and DORA emission.

The **Work ID** is the integration contract, threading through: branch name → commit message → PR title → pipeline env var → DORA event — a complete audit trail from code change to deployment.

Three AI agents augment the ecosystem: **PR Reviewer** (Amazon Q) for automated code review on every PR; **Spec Validator** (AWS Kiro) for design validation before code is written; **DORA Analyst** for converting event streams into actionable insights.

```
[Engineer] → devex branch FIN-123 → git hooks → PR
                                     ↓
[Framework] generates ↔ PR Pipeline → sandbox deploy
                                   → DORA event → CloudWatch
                                     ↓
[DORA Analyst] ← [REDACTED] event stream
```

2. Homologation — ensuring 10+ teams adopt the ecosystem

a) Convention over configuration

devex init generates everything a team needs in one command: .devex/config.yaml, git hooks, and stack detection. The path of least resistance is the golden path.

b) Installable from git — no registry required

```
uvx --from git+https://github.com/luicruz01/devex-platform#subdirectory=cli devex
pnpm add github:luicruz01/devex-platform#main --filter @luicruz01/devex-framework
```

Zero infrastructure to maintain. Works in air-gapped environments. Version pinning via git SHA.

c) Escape hatches without forking

LambdaServiceConstruct accepts additional environment variables via environment prop and runtime overrides via timeout prop — teams extend without modifying platform constructs.

d) Context engineering

Every repo gets .cursor/rules/ and .kiro/steering/ files. New engineers receive platform guidance from their IDE and spec-validation agents — no onboarding sessions required.

3. Scalability — avoiding platform team bottleneck

a) Inner-source model

Teams propose changes via GitHub Issues ([PROPOSAL] prefix). Platform team approves proposals, not implementations. Teams own their PRs.

b) AI agents as first-level support

Amazon Q reviews every PR before a human does. Kiro validates designs before code is written. The platform team only sees issues AI could not resolve — security escalations, schema changes, architectural disputes.

c) Typed escape hatches

LambdaServiceConstruct accepts extension props today. PipelineConfig is designed for pre/post hook extension in a future version — tracked as a known gap.

4. Shift-Left Strategy

Defect detection moves earlier through four layers:

- **Layer 1 — Design time.** Kiro steering files validate architecture decisions before a single line of code is written.
- **Layer 2 — Commit time.** pre-commit hook runs `devex check` — Work ID validation, lint, commit format. Defects caught in seconds.
- **Layer 3 — Push time.** pre-push hook runs the full unit test suite locally. Failures caught before reaching GitHub.
- **Layer 4 — PR time.** PR Pipeline runs tests, API contract validation, and sandbox deploy. Amazon Q reviews for security and convention violations.

A defect introduced at 9:00 AM is caught by 9:01 AM (pre-commit), not at 2:00 PM when a reviewer finds it in GitHub.

Key Technical Decisions

Decision	Choice	Rationale
Branching	Trunk-based + GitHub Environments	Eliminates long-lived branches; environment protection handles approvals
Pipeline gen	TypeScript template literals	Predictable, readable YAML output; no external YAML library dependency
DORA telemetry	Structured JSON to stdout	CloudWatch ingests from logs; same schema across all stacks
Distribution	git-based (uv + pnpm)	No registry infrastructure; version pinning via git SHA
Work ID	Configurable regex pattern	Works with Jira, Linear, GitHub Issues without platform team changes

What this PoC Demonstrates vs Production Gaps

Demonstrated	Production gaps (known, intentional for PoC scope)
CLI installable via <code>uvx</code>	AWS OIDC roles not provisioned (cdk synth, not cdk deploy)
Framework generates valid GitHub Actions YAML	Amazon Q integration is architectural — not wired to live PRs
CDK synth passes with <code>LambdaServiceConstruct</code>	DORA Analyst agent designed, not implemented
DORA events emitted with correct schema	No package registry — git-based install only
Context layer (Cursor rules + Kiro steering)	PipelineConfig pre/post hook extension not yet implemented
Work ID enforced locally and in PR pipeline	<code>work_id</code> uses 'N/A' in <code>init/check</code> — intentional: no Work ID exists at setup time